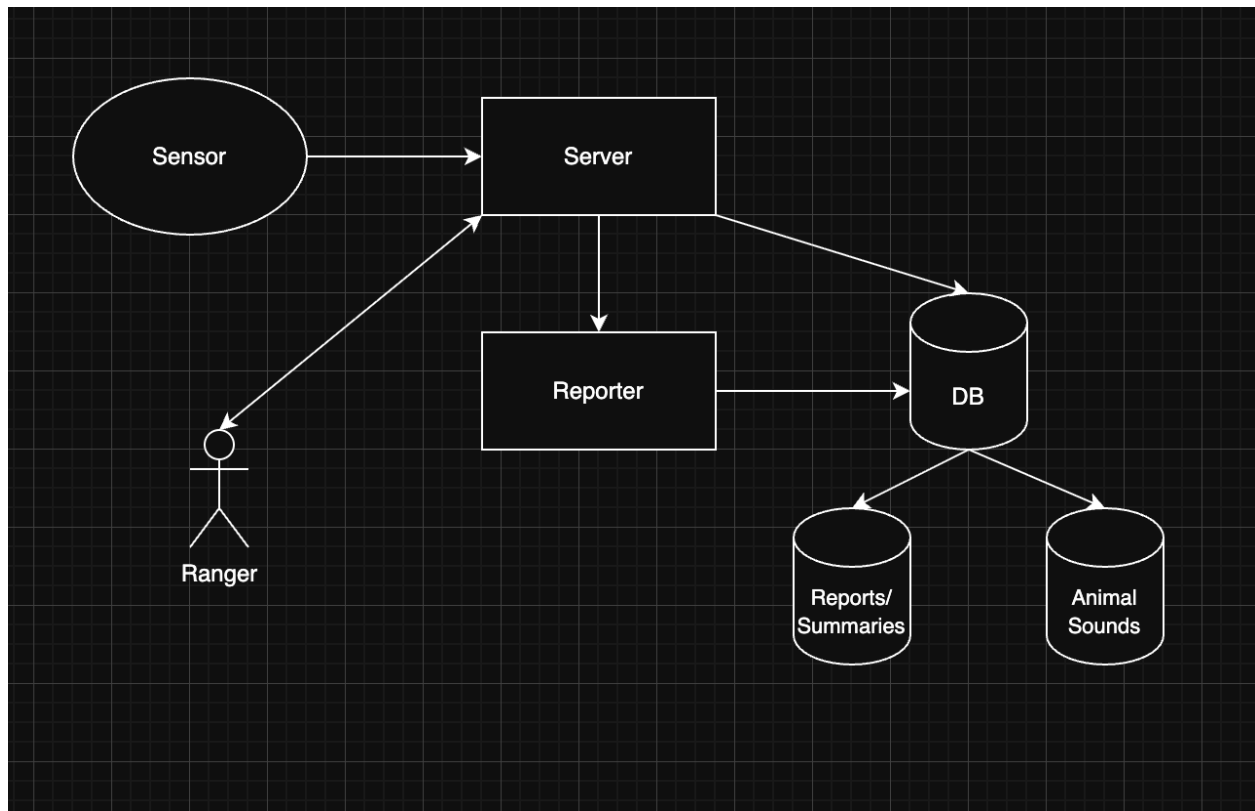


Updated SWA



<https://drive.google.com/file/d/15jdYijjVXwDU3NCKZe5S-P7oC7a9vrh1/view?usp=sharing>

We choose to split all our data into 2 databases. Our mountain lion detection system is not very complex. There are really 2 major components. The storage of the animals sounds that are recorded on the sensors and report summaries generated either by the rangers or when data becomes more than 30 days old. With how simplistic our system is we believe SQL to be the best choice of database management. With our data transforming in a summary after 30 days scalability can be handled with ease. However, if other parks want to implement our system, SQL will make that scalability slightly harder but still completely doable.

Data Management Strategy: SQL

We choose SQL as it is the best fit to our Mountain Lion Detection System. We need to ensure data integrity and manage strong relationships between key pieces of information, such as linking an alert to a specific sensor and then an animal

classification. SQL seems to be the easiest to implement and reliable to handle these needs.

Database Design Decisions

We will use a Single Physical SQL Database Server to manage two distinct logical groupings of data:

1. Daily Operations Data (Alerts)

- **Database:** Animal Sounds/Alerts
- **Purpose:** To manage the short term details. This is where every single sound detection event lands.
- **Data Stored:** Raw data (location, time, sound strength, animal, animal confirmation).
- **Time:** We keep this data for 30 days.
- **Why?** We need this data for immediate action (like a ranger classifying a fresh alert). We delete it after a month because sound data is too large to keep long term.

2. Long-Term Reporting Data (Summaries)

- **Database:** Report Summaries
- **Purpose:** To keep a history of all sounds and provide further analysis if needed.
- **Data Stored:** Low volume summarized data
- **Time:** We keep this data for a year.
- **Why?** We create summaries from the daily alert data before it's deleted. This keeps the data load manageable for long term storage and makes generating annual reports easier.

Logical Data Model

The system is broken down into five tables to ensure the data is organized, auditable, and easily queried:

- **Sensors:** Stores metadata for each physical sensor.
 - **Key Data:** sensorID, location (GPS), type.
 - **Relationship:** One sensor generates many alerts.

- **Rangers:** Stores information about the personnel using the system.
 - Key Data: rangerID.
 - Relationship: One ranger provides many classifications.
- **Alerts:** The primary table for detections.
 - Key Data: alertID, sensorID, timestamp, strength, location.
 - Relationship: One alert receives a classification.
- **RangerClassifications:** The record of a ranger's action on an alert.
 - Key Data: classificationID, alertID, rangerID, classification ('definite,' 'false,' etc.).
 - Relationship: Links the classification back to the specific Alert and Ranger.
- **ReportSummaries:** Stores the aggregated data for long-term analysis. (1-Year Retention)
 - Key Data: summaryID, monthYear, location, totalAlerts, definiteML_count.
 - Relationship: Standalone table derived from the other tables.

Alternatives and Tradeoffs

We chose SQL as the most convenient choice for our project:

- **Alternative: NoSQL (MongoDB, Redis, etc)**
 - **Data Organization:** Data is stored as JSON documents. Alerts and Reports would exist in separate collections.
 - **Tradeoffs:** No-SQL does offer better scalability and it's easier to add new data fields later. However, its increased code complexity adds risk to our infrastructure and possible data errors. In addition, our data is already very simplistic and adding it is unnecessary to increase our complexity. Lastly, we already have a limited number of fields, and do not envision adding more to our system later.